

Chapter 24: Change Detection

1. Overview

Angular is a data-binding framework. You write `{{ user.name }}` in a template, mutate `user.name` in a `setTimeout`, and the DOM updates — with no manual `render()` call anywhere in your code. Something has to notice that state changed and push the new value into the DOM. That "something" is **change detection (CD)**.

Change detection is Angular's mechanism for synchronizing the internal component state (your TypeScript class properties) with the rendered DOM. Conceptually it is the opposite problem to what frameworks like React solve with a virtual DOM diff: Angular does not diff two trees of `vnodes`. Instead, for every component it compiles a per-component **update function** at build time that knows exactly which DOM nodes and attributes depend on which expressions. Running change detection means walking the component tree and, for each component, executing that generated function to re-evaluate bindings and imperatively patch the DOM if a value changed.

Why does Angular need this at all, rather than just letting you call `render()` yourself? Because plain JavaScript gives no hook for "a property changed." Angular's answer historically was Zone.js: monkey-patch every async API (`addEventListener`, `setTimeout`, `Promise.then`, XHR callbacks, etc.) so that after any of them runs, Angular knows "something might have changed, time to check." That's why almost every trigger of change detection you'll be asked about in an interview — click handlers, HTTP responses, timers, promises — is really just "an async task completed inside Angular's zone, so Angular scheduled a CD pass afterward." (The deep mechanics of Zone.js patching and zoneless Angular are covered in Volume 3; this chapter gives you the mental model and the APIs you use day to day: `Default` vs `OnPush`, the tree walk, and `ChangeDetectorRef`.)

This chapter builds the mental model interviewers actually probe for: what triggers a check, what a check does, why `OnPush` exists and how it changes the walk, and the manual escape hatches (`markForCheck`, `detectChanges`, `detach`, `reattach`) you reach for when the automatic system isn't enough or is too expensive.

2. Core Concepts

2.1 What "a change detection pass" actually does

A CD pass is a **single, synchronous, top-down, depth-first walk of the component tree**, starting from the root component. For each component, Angular runs its generated `refreshComponent` logic, which:

1. Re-evaluates every interpolation and property binding in that component's template (`{{ }}`, `[prop]`, `[attr.x]`, `[class.x]`, `[style.x]`).
2. Compares each new value to the previously stored value (stored on the component's `LView`, a plain array of "current binding values").
3. If a value differs (checked with `!==`, i.e., reference/identity comparison for objects, value comparison for primitives), it imperatively updates the corresponding DOM property/attribute.
4. Recurses into child components in the order they appear in the template.

This is **one pass**, not a loop that keeps re-checking until stable (that was AngularJS's dirty-checking digest loop, which could run multiple iterations). Angular deliberately does a single pass and throws `ExpressionChangedAfterItHasBeenCheckedError` in dev mode if a second pass would produce a different value (see §5).

2.2 Why traversal is top-down and unidirectional

Angular assumes (and enforces, in dev mode) **unidirectional data flow**: data flows from parent to child via `@Input()` bindings, checked once per pass, top-down. A child is never allowed to silently push a new value back up into an already-checked ancestor during the same pass — that would require re-walking the ancestor, which is exactly what causes `ExpressionChangedAfterItHasBeenCheckedError`.

Because the walk is top-down and a parent is always checked before its children, by the time a child component is checked, its `@Input()` bindings have already been refreshed with the parent's latest values.

2.3 Default (`ChangeDetectionStrategy.Default`) vs `OnPush`

Default strategy: every component is checked on every CD pass, unconditionally. Angular doesn't try to be clever — it walks the entire tree from the root down, every time, regardless of whether that particular component could possibly have changed. This is simple and always correct, but wasteful in large trees where most components rarely change.

OnPush strategy (`changeDetection: ChangeDetectionStrategy.OnPush` in `@Component`): Angular still visits the component during the tree walk (it doesn't skip over it structurally), but it **skips actually refreshing bindings for that component's subtree unless one of these is true**:

- One of its `@Input()` properties received a **new reference** (`!==` the previous value) since the last check. Angular compares by identity, so mutating a nested field of an object/array input does **not** count — the reference is unchanged.
- An event bound in that component's own template fired (e.g., `(click)="onClick()"` in the component's own template, or a child's template — see below), including custom `@Output()` EventEmitters.
- Someone explicitly called `markForCheck()` (or `detectChanges()`) on that component's `ChangeDetectorRef`.
- The `async` pipe used in the template emitted a new value (it calls `markForCheck()` internally).

Internally, marking "dirty" sets a flag on the component's `LView` (conceptually `CheckAlways` for Default components, and a `Dirty/HasChildViewsToRefresh` bit that propagates upward for `OnPush` components — see §4). When Angular's tree walk reaches an `OnPush` component whose dirty flag is not set and whose inputs didn't change reference, it **skips checking that component and its entire subtree** — none of its descendants get checked, `OnPush` or not, because the parent's binding refresh is what would have fed them anyway.

This is the single most important interview fact about `OnPush`: **it doesn't just skip itself, it prunes the whole subtree**, which is exactly why `OnPush` is a genuine performance strategy, not just a hint.

2.4 How bindings actually get refreshed

For each binding in a template, the Angular compiler (via Ivy) generates instructions in the component's factory that, at runtime:

- Read the current value of the bound expression.
- Compare it to the value stored from the previous check (kept in the `LView` — an internal per-instance array Ivy uses to store binding state, providers, and more).
- If different, call the appropriate renderer method (`setProperty`, `setAttribute`, `addClass`, etc.) to push the change into the actual DOM node, or re-render the text node for interpolations.

This means there is no "vdom diff" step — the comparison is against the last known scalar/reference value per binding slot, and the DOM write is direct and targeted (only the specific attribute/property that changed is touched).

2.5 `ChangeDetectorRef` — the manual API

Every component/directive can inject `ChangeDetectorRef` to interact with CD manually:

Method	What it does
<code>markForCheck()</code>	Walks up the ancestor chain from this component, marking every ancestor (up to the root) as needing to be checked on the next CD pass. Does not run CD itself — it just ensures that the next time CD runs (e.g., triggered by an event elsewhere), this component won't be skipped. This is the standard way to tell Angular "an OnPush component's state changed even though no input reference changed and no template event fired" (e.g., a WebSocket callback, a manual RxJS subscription outside the async pipe).
<code>detectChanges()</code>	Synchronously runs change detection right now , starting at this component and walking down into its children (not up). Used to force an immediate, local refresh — e.g., after manually mutating state outside Angular's zone, or in tests.
<code>detach()</code>	Removes this component (and its subtree) from the CD tree. Angular will no longer check it automatically at all, on any pass, regardless of strategy, until you <code>reattach()</code> it or call <code>detectChanges()</code> on it explicitly.
<code>reattach()</code>	Re-inserts the component back into the normal CD tree walk, resuming automatic checks according to its strategy.
<code>checkNoChanges()</code>	Runs a verification pass without updating the DOM, used internally by Angular in dev mode to detect <code>ExpressionChangedAfterItHasBeenCheckedError</code> .

`detach()` / `reattach()` are the tool for **high-frequency update scenarios**: e.g., a live chart or a component driven by `requestAnimationFrame` at 60fps where you don't want Angular's tree walk touching it at all — you manage exactly when to render by calling `detectChanges()` yourself on your own schedule (e.g., throttled to 10fps).

2.6 What triggers a change detection pass in the first place

With Zone.js (the default, non-zoneless setup), Angular patches every asynchronous browser API. After any of the following completes, Zone.js notifies Angular ("the zone became stable" / `onMicrotaskEmpty` or `onInvoke` completes) and Angular runs `ApplicationRef.tick()`, which walks the whole component tree from the root:

- **DOM events**: click, input, keyup, scroll, etc. — anything bound in a template with

`(event)="handler()"`, or attached via `addEventListener` since Zone.js patches that too.

- **HTTP responses:** `HttpClient` calls `resolve` via XHR/fetch callbacks, which are zone-patched.
- **Timers:** `setTimeout`, `setInterval`, `requestAnimationFrame`.
- **Promises:** `.then()` / `.catch()` / `async` - `await` continuations (patched as microtasks).
- **Other async browser APIs:** `MutationObserver`, `IntersectionObserver`, WebSocket message events, drag-and-drop events, etc.

Angular does **not** run CD on its own timer or polling loop — it is purely event-driven, riding on top of Zone.js's ability to know when any patched async callback has run. If you run code entirely outside Angular's zone (`NgZone.runOutsideAngular(...)`) — e.g., a `setInterval` used to drive a canvas animation — Angular will never automatically know to check bindings for that state change; you must manually call `NgZone.run(...)` or `ChangeDetectorRef.detectChanges()` / `markForCheck()` to surface it.

3. Code Examples

```
import {
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  Input,
  NgZone,
  OnDestroy,
  OnInit,
} from '@angular/core';
import { Subscription, interval } from 'rxjs';

/**
 * A high-frequency live-updating widget (e.g., a stock ticker or telemetry
 * gauge) that receives updates far faster than the UI needs to render them.
 *
 * Strategy:
 * - OnPush, so Angular's normal event/input-driven CD passes are cheap
 *   (this component is skipped unless we explicitly ask for a check).
 * - The high-frequency data stream runs OUTSIDE Angular's zone so it never
 *   triggers a global tick() on every single value.
 * - The component is detach()-ed from the CD tree entirely, and we drive
 *   our own throttled detectChanges() calls (e.g., 10 times/sec instead
 *   of 500 times/sec).
 */
@Component({
  selector: 'app-live-gauge',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <div class="gauge">
      <span class="label">{{ label }}</span>
      <span class="value">{{ latestValue | number: '1.2-2' }}</span>
      <span class="samples">({{ sampleCount }} samples)</span>
    </div>
  `,
})
```

```

export class LiveGaugeComponent implements OnInit, OnDestroy {
  @Input() label = 'Signal';

  latestValue = 0;
  sampleCount = 0;

  private rawUpdatesSub?: Subscription;
  private renderLoopSub?: Subscription;

  constructor(
    private readonly cdr: ChangeDetectorRef,
    private readonly zone: NgZone,
  ) {}

  ngOnInit(): void {
    // Detach: Angular's normal tree walk will never touch this component's
    // bindings again until we explicitly reattach() or call detectChanges().
    this.cdr.detach();

    this.zone.runOutsideAngular(() => {
      // Simulates a high-frequency data source (e.g., a WebSocket at ~500Hz).
      this.rawUpdatesSub = interval(2).subscribe(() => {
        this.latestValue = Math.random() * 100;
        this.sampleCount++;
        // NOTE: we do NOT call markForCheck/detectChanges here – that would
        // defeat the point of detaching. We just update plain fields.
      });

      // Our own render cadence: refresh the DOM at a sane 10fps instead of
      // on every single sample.
      this.renderLoopSub = interval(100).subscribe(() => {
        // detectChanges() runs CD synchronously for THIS component and its
        // children only, ignoring the detached state (detach only affects
        // the automatic tree walk, not explicit calls).
        this.cdr.detectChanges();
      });
    });
  }

  /** Called from a template button: "pause" the gauge entirely. */
  pause(): void {
    this.cdr.detach();
  }

  /** Called from a template button: resume normal OnPush behavior. */
  resume(): void {
    this.cdr.reattach();
    this.cdr.markForCheck();
  }

  ngOnDestroy(): void {
    this.rawUpdatesSub?.unsubscribe();
    this.renderLoopSub?.unsubscribe();
  }
}

```

```
}  
}
```

```
import {  
  ChangeDetectionStrategy,  
  ChangeDetectorRef,  
  Component,  
  Input,  
} from '@angular/core';  
  
/**  
 * A typical OnPush component that mutates a nested field of an @Input()  
 * object in response to a WebSocket push arriving OUTSIDE any Angular-  
 * triggered flow (e.g., a raw `new WebSocket(...).onmessage` handler that  
 * Zone.js may or may not have patched depending on config). Because the  
 * @Input() reference itself never changes, OnPush would normally never  
 * re-render – markForCheck() is the fix.  
 */  
@Component({  
  selector: 'app-order-status',  
  changeDetection: ChangeDetectionStrategy.OnPush,  
  template: `  
    <p>Order #{{ order.id }} – status: {{ order.status }}</p>  
  `,  
})  
export class OrderStatusComponent {  
  @Input() order!: { id: string; status: string };  
  
  constructor(private readonly cdr: ChangeDetectorRef) {}  
  
  /** Invoked by a raw socket callback, not by an Angular-zone-aware API. */  
  onStatusPush(newStatus: string): void {  
    this.order.status = newStatus; // mutation, not a new reference  
    this.cdr.markForCheck(); // tell Angular: check me (and ancestors) next pass  
  }  
}
```

4. Internal Working

4.1 The tree walk, precisely

When `ApplicationRef.tick()` runs, Angular calls `detectChanges` (internally `refreshView`) on the root `LView` and recurses. For each component `LView` visited:

1. If the component's strategy is `Default`, or the `LView`'s `Dirty / CheckAlways` flag is set, Angular refreshes its bindings (re-evaluates expressions, patches DOM on change) and then **recurses into every child view**, regardless of the child's own dirty state — the child still gets its turn to decide whether to actually do work.
2. If the component is `OnPush` and **not** marked dirty and received no new `@Input()` reference this pass, Angular **does not refresh its bindings and does not descend into its children at all** — the whole subtree rooted there is pruned from this pass.

3. Template event bindings (`(click)="..."`) are wired at compile time to also mark their own component's `LView` dirty before invoking the handler, which is why clicking a button inside an `OnPush` component always triggers at least that component (and its ancestors, since the pass is already in progress and any ancestor state the handler mutates will be visible) to be checked — you don't need `markForCheck()` just to react to your own template's DOM events.

4.2 Mapping `ChangeDetectorRef` to `LView` flags

Internally, each component instance has a `LView` (an array-like data structure holding binding slots, providers, `TView` reference, and a set of flags). The flags most relevant here (conceptually, exact bit names are an implementation detail across versions):

- `markForCheck()` — sets a `Dirty` flag on this `LView`, then walks up through `parent LView` references setting a "has dirty descendant" flag on every ancestor up to the root. This is why a `markForCheck()` deep in the tree still causes the walk starting from the root to reach it: each ancestor's "descendant needs checking" flag keeps the walk from skipping over that whole branch, even though the ancestor's own bindings may not have changed.
- `detectChanges()` — bypasses the dirty-flag check entirely and directly invokes `refreshView` on this component's `LView` and descends into children immediately, synchronously, regardless of strategy or flags. It doesn't set any ancestor flags — it's a one-off local refresh, not a schedule request.
- `detach()` — sets a flag on the `LView` that tells the parent's traversal logic to skip this view entirely during the automatic walk (as if it were structurally removed from CD's perspective), independent of the `OnPush/Default` distinction. The component is still fully alive (bindings, DOM, subscriptions) — only automatic checking is suspended.
- `reattach()` — clears that flag, restoring normal traversal (`Default` = always checked; `OnPush` = checked only when dirty/new-inputs, as usual).

4.3 Why `async` pipe "just works" with `OnPush`

The `AsyncPipe` injects `ChangeDetectorRef` for the component whose template it's used in, subscribes to the `Observable/Promise` you pass it, and on every emission calls `markForCheck()` before returning the new value for interpolation. That's the entire trick — it's not magic, it's the exact same manual API described above, wired up for you.

5. Edge Cases & Gotchas

OnPush + mutated objects never re-render. If `@Input() config: Config` and a child does `this.config.enabled = true` (or the parent mutates the same object it already passed down), the reference never changes, so an `OnPush` component checking only that input will never notice. Fix: treat inputs as immutable and always pass new references (`spread/ object.assign/ structuredClone`), or explicitly call `markForCheck()` after a deliberate mutation.

`async` pipe automatically marks for check. As covered above — this is precisely why `| async` is the idiomatic pairing with `OnPush` + `Observables/Signals`-based state: you get automatic, minimal-surface change notification without manually wiring `markForCheck()` everywhere.

Detecting changes from outside Angular / third-party libraries. Libraries that attach raw DOM listeners or use their own scheduling (e.g., a canvas library's internal `requestAnimationFrame`, a non-Angular-aware WebSocket client, or code deliberately run via `NgZone.runOutsideAngular()`) won't trigger Angular's zone hooks. Angular has no idea state changed. You must bridge back in explicitly: either wrap the callback in `NgZone.run(() => { ...; this.cdr.markForCheck(); })`, or call `ChangeDetectorRef.detectChanges() / markForCheck()` directly if you don't want to re-enter the zone (avoiding the overhead of a full-tree `tick()` for a highly localized update).

ExpressionChangedAfterItHasBeenCheckedError. In development mode, after each CD pass Angular runs an extra verification pass (`checkNoChanges`) comparing the bindings again; if any value differs from what was just rendered, it throws this error. Classic causes:

- A child's `ngAfterViewInit` (or a parent method invoked during the same pass) mutates a value the parent template already displayed **during the same tick**, so the parent's binding computed one value, and by verification time it's different.
- Binding directly to a getter that returns a new object/array every call (`[data]="getData()"` where `getData()` returns `{...}` fresh each time) — the "value" is different every single time it's read, guaranteeing an eventual mismatch.
- This error **only throws in dev mode** — production builds skip the verification pass, so the same bug in production silently renders a stale value for one extra frame instead of crashing. Treat the dev-mode error as a real bug signal, not noise to suppress.
- Common fixes: move the mutation to happen before the parent's own check (e.g., in a resolver, in `ngOnInit`, or via `setTimeout / Promise.resolve().then()` to defer to the *next* CD cycle), or explicitly call `cdr.detectChanges()` synchronously right after the mutation so both passes see the same, final value.

detach() silently "freezing" a component after navigation/reuse. If you `detach()` a component instance and it's later reused via `*ngIf / router` state without `reattach()`, it can appear to "stop updating" mysteriously — always pair `detach()` with a clear `reattach()` (e.g., in `ngonDestroy` cleanup logic isn't relevant, but on whatever lifecycle re-enables the widget).

Marking ancestors, not the whole tree. `markForCheck()` only guarantees the path from this component up to the root gets walked on the *next* pass — it does not itself schedule that pass. If nothing else triggers a tick afterward (e.g., you called it from code running entirely outside the zone with no further zone-triggering event), the DOM won't actually update until some other trigger fires `tick()`. In practice, pair manual `markForCheck()` calls made outside the zone with `ApplicationRef.tick()` or re-entering the zone.

6. Interview Questions & Answers

Q1. What is change detection in Angular, in one sentence?

It's the process by which Angular walks the component tree and re-evaluates template bindings to keep the DOM in sync with the current state of component class properties, imperatively patching only what changed.

Q2. Why does Angular need change detection at all — why can't the DOM just "know" state changed?

Interviewer intent: checking whether you understand the fundamental problem (no native reactivity in plain JS objects) rather than just reciting API names.

Plain JavaScript object property assignment (`this.x = 5`) doesn't emit any event or notification by default — there's no built-in observer mechanism for arbitrary object mutation. Angular needs some way to know "maybe something changed, go check the bindings." Historically it solved this with Zone.js monkey-patching all

async browser APIs so it can run a check after any event/timer/promise callback completes. Newer Angular (signals) solves it more precisely via explicit reactive primitives that track their own dependents, reducing reliance on the "check everything, always" fallback.

Q3. What's the difference between `ChangeDetectionStrategy.Default` and `OnPush`?

Default checks the component unconditionally on every CD pass. `OnPush` skips checking the component (and prunes its entire subtree from that pass) unless: an `@Input()` got a new reference, a DOM event fired in that component's own template, `markForCheck()` / `detectChanges()` was called explicitly, or the `async` pipe emitted.

Q4. If an `OnPush` component's `@Input()` is an object and a grandchild mutates a nested property on it, will the `OnPush` component re-render?

No. `OnPush` compares input bindings by reference (`!==`). A mutation of a nested field doesn't change the outer object's reference, so Angular doesn't consider the input "changed," and the subtree isn't re-checked automatically. You'd need immutable updates (new reference each change) or an explicit `markForCheck()`.

Q5. What does `markForCheck()` actually do internally?

It doesn't run change detection itself. It walks up the ancestor chain from the current component's view, setting a "dirty"/"has dirty descendant" flag on every ancestor up to the root. This ensures that the *next* time a CD pass runs (triggered by whatever fires it — a click elsewhere, a timer, etc.), the walk won't skip over this branch even though the ancestors' own bindings haven't changed.

Q6. How is `detectChanges()` different from `markForCheck()`?

`markForCheck()` schedules/enables a future check (sets flags, doesn't check now). `detectChanges()` runs change detection **synchronously, immediately**, for this component and its descendants, regardless of dirty flags or strategy — it's an on-demand local CD pass, not a scheduling hint.

Q7. What do `detach()` and `reattach()` do, and when would you use them?

Interviewer intent: probing whether you know a real performance technique beyond "just use `OnPush`," typically for high-frequency-update UI (charts, gauges, real-time feeds).

`detach()` removes a component's view from the automatic CD tree walk entirely — Angular will never check it on its own until `reattach()` is called, or you manually call `detectChanges()` on it (which still works even while detached). This is useful when you want full manual control over render cadence — e.g., a canvas/chart component receiving updates 100x/second, where you throttle actual re-renders to 10x/second via your own `setInterval` / `requestAnimationFrame` loop calling `detectChanges()`, rather than letting Angular's zone-driven tick fire on every single update.

Q8. Name the common triggers of a change detection pass.

DOM events bound in templates (and any zone-patched `addEventListener`), HTTP responses via `HttpClient` (XHR/fetch callbacks are zone-patched), timers (`setTimeout` / `setInterval` / `requestAnimationFrame`), promise/microtask continuations (`.then()`, `async` / `await`), and other patched async browser APIs (`MutationObserver`, WebSocket message events, etc.). All of these work by Zone.js patching the API so Angular's `NgZone` knows when a task completed and can call `ApplicationRef.tick()` afterward.

Q9. If I run code inside `NgZone.runOutsideAngular()`, will change detection still run automatically for that code's side effects?

No. Code run outside the Angular zone means any timers/promises/events it schedules also execute outside the zone, so Zone.js never notifies Angular that a task completed, and `tick()` never gets scheduled because of it. You must manually re-enter (`NgZone.run(...)`) or call

`ChangeDetectorRef.markForCheck()` / `detectChanges()` yourself to surface state changes to the view.

Q10. Why is the CD tree walk described as "top-down, single-pass, unidirectional"? What breaks this assumption?

Angular checks parents before children in one direction, on the assumption that data flows parent→child via inputs and a child never needs to reach back and change something in an already-checked ancestor within the same pass. If a child's lifecycle hook (e.g., `ngAfterViewInit`) or a template reference mutates a value the parent already rendered in that same pass, the parent's previously-computed value is now stale — that's precisely what triggers `ExpressionChangedAfterItHasBeenCheckedError` in dev mode.

Q11. Explain `ExpressionChangedAfterItHasBeenCheckedError` — cause, when it fires, and how you'd fix it.

Interviewer intent: this is a very common real-world debugging question; they want to see you can diagnose it, not just name it.

After a full CD pass, Angular dev mode runs a second verification pass (`checkNoChanges`) comparing each binding's value again; if any binding now evaluates differently than what was just rendered, it throws this error. Typical cause: a value gets mutated *during* the same tick after the component displaying it was already checked — e.g., a child's `ngAfterViewInit` sets `@Input()`-derived state on the parent, or a template binds directly to a getter/function call that returns a fresh object every invocation. It only throws in dev — production silently renders the stale value for one extra frame. Fixes: avoid mutating state that's already been rendered within the same pass (move the update earlier, e.g., `ngOnInit`, a resolver, or defer it to the next tick via `Promise.resolve().then()` / `setTimeout`), or explicitly call `detectChanges()` right after the mutation so the same pass's verification sees the final value.

Q12. How does the `async` pipe interact with OnPush change detection?

It subscribes to the Observable/Promise on your behalf and, on every new emission, calls `markForCheck()` on the host component's `ChangeDetectorRef` before making the emitted value available for binding. It also automatically unsubscribes on destroy. This is why `| async` is the canonical partner for OnPush — you get correct, minimal, automatic re-rendering without writing manual `markForCheck()` calls throughout your subscription callbacks.

Q13. In an OnPush component tree, if the root is Default and a deeply nested grandchild is OnPush with no changed inputs, does Angular still visit that grandchild during a tick?

Angular's traversal reaches the grandchild's parent in the walk, but if the grandchild's dirty flags aren't set and no input reference changed, Angular skips refreshing its bindings and does not descend into its own children — that entire subtree is pruned from the pass. It's "visited" only in the sense that the walk arrives at that point in the tree structure; no binding evaluation or DOM patching happens for it or anything beneath it.

Q14. Does calling `ChangeDetectorRef.detectChanges()` on a component check its ancestors too?

No. `detectChanges()` only refreshes the component it's called on and recurses **downward** into its children — it never walks upward to ancestors. If an ancestor's own bindings also need refreshing because of the same state change, you need `markForCheck()` (which walks up) instead, or a separate mechanism to trigger a full `tick()`.

Q15. Is change detection the same thing as Angular's "digest cycle" from AngularJS?

No, and this distinction matters. AngularJS's digest loop performed dirty-checking by re-evaluating watched expressions repeatedly (potentially many iterations) until no value changed between iterations, across the whole scope hierarchy, with no compiled binding metadata. Angular's Ivy change detection instead compiles per-component update functions ahead of time, does a single top-down pass per tick (with only a lightweight dev-mode verification pass, not a repeated loop), and supports pruning entire subtrees via OnPush — fundamentally faster and more predictable than AngularJS's approach.

7. Quick Revision Cheat Sheet

- **CD pass** = one top-down, depth-first walk of the component tree from the root, re-evaluating bindings and patching only changed DOM values (no vdom diffing — direct, compiled binding checks).
- **Default:** every component checked, every pass, unconditionally.
- **OnPush:** component (and its whole subtree) skipped unless: new `@Input()` reference, own-template DOM event fired, `markForCheck()` / `detectChanges()` called, or `async` pipe emitted.
- `markForCheck()`: sets dirty flags up the ancestor chain; schedules future checking, doesn't check now.
- `detectChanges()`: runs CD synchronously, right now, for this component + children only; ignores strategy/dirty flags; doesn't touch ancestors.
- `detach()` / `reattach()`: fully remove/restore a view from the automatic tree walk — use for manual render-cadence control (high-frequency widgets).
- **Triggers:** DOM events, HTTP callbacks, timers, promises/microtasks, other zone-patched async APIs — all via Zone.js noticing a patched task completed, then calling `ApplicationRef.tick()`.
- `NgZone.runOutsideAngular()`: opts code out of auto-triggering CD; must manually `NgZone.run()` or call `markForCheck()` / `detectChanges()` to surface changes.
- **OnPush pitfall:** mutating a nested field of an `@Input()` object doesn't change its reference → no re-render. Use immutable updates or `markForCheck()`.
- **async pipe:** automatically calls `markForCheck()` on each emission — the standard OnPush + Observable pairing.
- `ExpressionChangedAfterItHasBeenCheckedError`: dev-mode-only; thrown when a second verification pass finds a binding value differs from what was just rendered, usually from mutating already-checked state within the same tick.

Created By - Durgesh Singh